

**HOCHSCHULE
HANNOVER**
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS
–
*Fakultät IV
Wirtschaft und
Informatik*

Convolution und ihre Anwendung in der digitalen Audiotbearbeitung

Julian Mueller und Tom Nir

Seminar-Arbeit im Studiengang „Angewandte Informatik“

17. Mai 2026



Autor 1: Julian Mueller
1813409
julian.mueller@stud.hs-hannover.de
Verfasste Seiten/Abschnitte:
Kapitel 1: Einleitung
Sektion 2.1: Abtasttheorem
Sektion 2.2: Zeit- und Frequenzdomäne
Sektion 2.3: Nyquist-Shannon Theorem
Sektion 3.2: Laufzeitanalyse
Kapitel 4: Implementierung der Convolution

Autor 2: Tom Nir
1821763
tom.nir@stud.hs-hannover.de
Verfasste Seiten/Abschnitte:
Sektion 2.4: Signale und Systeme
Sektion 3.1: Convolution
Sektion 3.3: LTI-Systeme in der realen Welt
Sektion 3.4: Techniken zur Messung von Raum-Impulsantworten

Prüferin: Toni Hänßgen
Abteilung Informatik, Fakultät IV
Hochschule Hannover
toni.haenssngen@hs-hannover.de

Selbständigkeitserklärung

Mit der Abgabe der Ausarbeitung erklären wir, dass wir die eingereichte Seminar-Arbeit selbständig und ohne fremde Hilfe verfasst, andere als die von uns angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Werken wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht haben.

Hannover, den 17. Mai 2026

1 Einleitung

In der Welt der digitalen Signalverarbeitung ist die Faltung (engl.: „Convolution“) ein mathematischer Vorgang, bei dem ein Eingangssignal mit einem weiteren Signal, einer sog. Impulsantwort gefaltet wird. Eine Impulsantwort ist hierbei als diskretes Audiosignal zu verstehen, das die Klangeigenschaften eines bestimmten Systems (bspw. Lautsprecherboxen, Räume, Filter, ... etc.) beinhaltet. Die Convolution beschreibt also mathematisch den Einfluss eines solchen Systems auf ein Eingangssignal.[[Lyo10](#), Kapitel 5]

In den folgenden Kapiteln werden folgende Aspekte der Convolution erläutert:

- Die mathematischen Grundlagen der Convolution diskreter Audiosignale
- Veranschaulichung verschiedener Systeme, die mithilfe der Convolution realisiert werden können
- Praktische Anwendung und die damit verbundenen Problemstellungen
- Gegenüberstellung: Naive Umsetzung der diskreten Convolution und Umsetzung mithilfe der *Fast Fourier Transformation* (kurz: FFT)
- Implementierung in Software zur digitalen Audiobearbeitung

Allerdings wird im Rahmen dieser Facharbeit nicht auf die mathematische Definition der schnellen Fourier Transformation eingegangen, weil dies den Rahmen der Seminararbeit sprengen würde. Stattdessen werden alle Techniken und Theoreme erläutert, die für das grundlegende Verständnis der Convolution notwendig sind.

2 Theoretische Grundlagen

2.1 Abtasttheorem

Die folgenden Veranschaulichungen und mathematischen Definitionen orientieren sich an den Darstellungen von Lyons [Lyo10, Kap. 1] und Smith [Smi97, Kap. 3]

Aus physikalischer Sicht sind Töne Luftdruckschwankungen, die sich in Form von Wellen durch ein Medium, wie bspw. der Erdatmosphäre ausbreiten. Solche Schwankungen lassen sich mithilfe von Wandlern wie Mikrofonen in stetige Wechselstromsignale übertragen:

[Lyo10, Kap. 1]:

$$y(t) = \sin(2\pi \cdot f \cdot t) \quad (2.1)$$

Problematisch hierbei ist, dass ein solches analoges Signal aufgrund seiner Stetigkeit eine überabzählbare Anzahl an potentiellen Messpunkten aufweist und sich dadurch unmöglich in Form einer Datenstruktur speichern lässt.

Um dieses Problem nun zu lösen und das Signal dennoch zu digitalisieren, wird eine Abtastung (engl.: *sampling*) in zeitlich gleichmäßigen Abständen vorgenommen:

2.2 Zeit- und Frequenzdomäne

Im vorherigen Abschnitt wurden Signale (sowohl stetige als auch diskrete) ausschließlich in der sog. *Zeitdomäne* betrachtet. Hier tritt das Signal in Form einer Welle in Erscheinung. Diese Darstellung ist aber hinsichtlich der darin enthaltenen Frequenzen wenig aufschlussreich. Anders ist es mit der *Frequenzdomäne* desselben Signals, was die folgende Grafik veranschaulicht:

Wie in Abbildung 2.1 illustriert, beschreibt die Zeitdomäne eines Signals (links) mit seiner horizontalen Achse die voranschreitende Zeit in n (*Samples*) und mit seiner vertikalen Achse die Momentanamplituden zum jew. Zeitpunkt einer Abtastung.

Die Frequenzdomäne hingegen zeigt das komplette Spektrum an Frequenzen, die das Signal beinhaltet. Man kann sich die Frequenzdomäne wie ein Prisma vorstellen, das einfallendes, weißes Licht klar sichtbar in seine farbigen spektralen Bestandteile zerlegt.

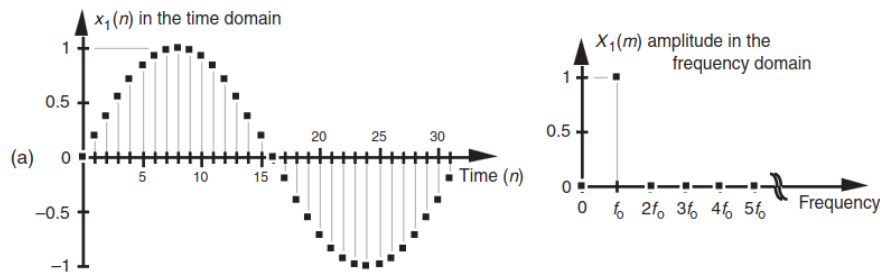


Abbildung 2.1: Links: Zeitdomäne eines Signals, Rechts: Frequenzdomäne desselben Signals [Lyo10, Kap. 1, S. 6]

Hier steht die vertikale Achse für die jeweilige Frequenzstärke, als positiver Amplitudenwert definiert. Die horizontale Achse repräsentiert die Frequenz in Hz (bei Audiosignalen wären das verschiedene Tonhöhen, die gleichzeitig und in verschiedenen Stärken erklingen). [Lyo10, Kap. 2]

Ein Signal in seiner Zeitdomäne lässt sich durch die *Diskrete Fourier Transformation* (kurz: *DFT*) in die Frequenzdomäne umwandeln:

$$X[m] = \sum_{n=0}^{N-1} x[n] e^{-i2\pi nm/N} \quad (2.2)$$

[Lyo10, Kap. 3]

2.3 Nyquist-Shannon Theorem

Wie in [Lyo10, Kap. 2] beschrieben, gibt es für jede Folge an abgetasteten Werten unendlich viele Möglichkeiten, dieses Signal zu interpretieren. Die ursprüngliche Frequenz des Signals wird periodisch und unendlich oft ober- und unterhalb der jeweiligen Frequenz gespiegelt (auch *Aliasing* genannt) [Lyo10, Kapitel 2]. Veranschaulicht in folgender Grafik:

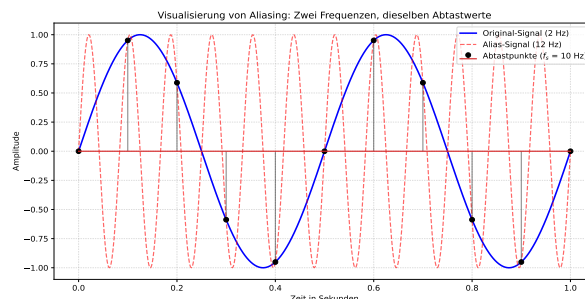


Abbildung 2.2: Neben den beiden abgebildeten Kurven könnten unendlich viele weitere Kurven über die Abtastwerte gelegt werden

Mathematisch genau lässt sich dieses Phänomen wie folgt beschreiben:

$$x[n] = \sin(2\pi \cdot f_0 \cdot n \cdot t_s) = \sin(2\pi \cdot (f_0 + k \cdot f_s) \cdot n \cdot t_s) \quad (2.3)$$

wobei:

f_0 : Ursprungsfrequenz in Hz

f_s : Abtastfrequenz in Hz

$n \in \mathbb{N}$: Index der Folge von abgetasteten Werten

$k \in \mathbb{Z}$: Faktor, der die unendlichen spektralen Wiederholungen beschreibt

Die Faktoren f_0 und $(f_0 + k f_s)$ führen also zum gleichen Ergebnis an Signalwerten.

Der mathematische Beweis dafür wird hier übersprungen, da es den Rahmen sprengen würde [Lyo10, vgl. Kap 2]. Um mit dieser Problematik umzugehen, bedient man sich der sog. *Nyquist-Frequenz* oder auch *Nyquist-Grenze*, die wie folgt definiert ist:

$$f_{Nyquist} = \frac{f_s}{2} \quad (2.4)$$

wobei f_s wieder für die Abtastfrequenz steht.

2.4 Signale und Systeme

In diesem Abschnitt werden einige für den weiteren Verlauf wichtige Begriffe definiert, insbesondere *Signal* und *System*, sowie die sogenannten Linearen Zeit-Invarianten Systeme.

Signale Ein Signal beschreibt, wie ein Parameter sich im Verhältnis zu einem anderen Parameter verändert. Signale übertragen Informationen über den Zustand oder das Verhalten eines physischen Systems. Mathematisch sind Signale Funktionen einer oder mehrerer unabhängiger Variablen. Die unabhängige Variable von Audiosignalen ist Zeit oder Frequenz [Smi97, Kap. 5]. Die unabhängige Variable kann stetig oder diskret sein, wobei stetige Signale oft als *analoge* Signale bezeichnet werden [OSB99, Kap. 2]. In Computergestützten Anwendungen werden ausschließlich diskrete Signale verarbeitet. Der Definitionsbereich Diskreter Signale sind die natürlichen Zahlen. Diskrete Signale sind also mathematisch betrachtet Folgen. Signale $S : \mathbb{N} \mapsto \mathbb{Z}$ heißen *digitale Signale* [OSB99, Kap. 2]. Stetige Signale werden mit der unabhängigen Variable in runden Klammern notiert: $x(t)$ für ein stetiges Signal x in Abhängigkeit von Zeit t . Diskrete Signale werden mit der unabhängigen Variable in eckigen Klammern notiert: $x[n]$ für ein diskretes Signal x , wobei n den Index des betrachteten Funktionswerts bezeichnet [OSB99, Seite 9].

Der Einheitsimpuls Ein besonderes diskretes Signal ist die *Einheits-Sample-Folge*. Sie ist definiert als die Folge

$$\delta[n] = \begin{cases} 0, & n \neq 0 \\ 1, & n = 0 \end{cases} \quad (2.5)$$

Jedes diskrete Signal lässt sich als eine Summe von skalierten und verzögerten Einheitsimpulsen darstellen [OSB99, Kap. 2, Seite 11]. Im weiteren Verlauf wird die Einheits-Sample-Folge etwas vereinfacht als *Einheitsimpuls* bezeichnet.

Systeme Ein System produziert als Antwort auf ein Eingangssignal ein Ausgangssignal [Smi97, Kap. 5]. Formal definiert [OSB99, Seite 16] ein diskretes System als eine Transformation oder einen Operator, welche(r) ein Eingangssignal $x[n]$ auf ein Ausgangssignal $y[n]$ abbildet. Die Funktion $T : x[n] \mapsto y[n]$ heißt *Transferfunktion* des Systems.

Lineare Zeit-Invariante Systeme (LTI) Besonders signifikant sind in der Signalverarbeitung Lineare Zeit-Invariante Systeme. Damit ein System *linear* heißt, muss es die beiden Eigenschaften

$$T(x_1[n] + x_2[n]) = T(x_1[n]) + T(x_2[n]) = y_1[n] + y_2[n] \quad (2.6)$$

und

$$T(a \cdot x[n]) = a \cdot T(x[n]) = a \cdot y[n] \quad (2.7)$$

erfüllen. Daraus resultiert die *Superposition* von Eingangssignalen:

$$x[n] = \sum_k a_k x_k[n] \Rightarrow y[n] = \sum_k a_k y_k[n] \quad (2.8)$$

Wobei $x[n]$ das Eingangssignal, $y[n]$ das Ausgangssignal, und a_k eine Folge von Konstanten bezeichnen [OSB99].

Ein System heißt *Zeit-Invariant*, wenn für jedes n_0 und jedes Eingangssignal x mit den Werten $x'[n] = x[n - n_0]$ das entsprechende Ausgangssignal y mit den Werten $y'[n] = y[n - n_0]$ erzeugt wird [OSB99].

Ist ein System sowohl linear als auch Zeit-Invariant, dann heißt es *lineares Zeit-Invariantes System (LTI-System)*. Die Auswirkungen von LTI-Systeme auf ein beliebiges Eingangssignal können vollständig durch ihre Impulsantwort (engl.: *Impulse Response*, kurz *IR*, siehe 3.1) ausgedrückt werden [OSB99]. Die Impulsantwort eines Systems ist selbst ein diskretes Signal und wird mit $h[n]$ notiert [OSB99, Kapitel 2, Seite 23].

3 Convolution in der digitalen Audiotbearbeitung

3.1 Convolution

In 2.4 wurde gezeigt, dass Lineare Zeit-invariante Systeme sich vollständig durch ihre Impulsantwort charakterisieren lassen. Hier werden Impulsantwort und die Convolution-Summe definiert.

[OSB99, Seite 11] formuliert zur Präzisierung der in 2.4 gezeigten Interpretation eines Signals als Summe skaliert und verzögerter Einheitsimpulse eine allgemeine Summenformel, mit der eine beliebige Folge $x[n]$ ausgedrückt wird:

$$x[n] = \sum_{k=-\infty}^{\infty} x[k]\delta[n-k] \quad (3.1)$$

Die mathematische Beschreibung der diskreten Convolution wird daraus wie folgt abgeleitet:

Sei $y[n]$ das Ausgangssignal eines LTI-Systems, resultierend aus einem Eingangssignal $x[n]$. Sei $h_k[n]$ die Antwort eines Systems auf einen Einheitsimpuls an Index $n = k$. Es gilt:

$$y[n] = T \left(\sum_{k=-\infty}^{\infty} x[k]\delta[n-k] \right). \quad (3.2)$$

Nach Superposition (Linearität) gilt auch:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]T(\delta[n-k]) = \sum_{k=-\infty}^{\infty} x[k]h_k[n]. \quad (3.3)$$

In Gleichung 3.3 ist $h_k[n]$ vom Laufindex der Summenfunktion abhängig. Es würden für die Berechnung der Summe also genauso viele System-Antworten benötigt, wie Summierungen ausgeführt werden. Unter Forderung der Zeitinvarianz des Systems fällt diese Eigenschaft weg:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]. \quad (3.4)$$

Die Gleichung 3.4 wird als *Convolution-Summe* bezeichnet [OSB99, Seite 22f]. Als kürzere Schreibweise für die Convolution zweier Signale x und h zu einem Ausgangssignal y dient die Schreibweise $y[n] = x[n] * h[n]$.

Das Prinzip kann anhand eines simplen Anwendungsfalls, dem Tiefpassfilter, veranschaulicht werden. Visualisierungen in diesem Abschnitt sind an die Grafiken von Richard Lyons [Lyo10, Kapitel 5] angelehnt. Tiefpassfilter können in der diskreten Zeitdomäne durch ein von [Lyo10, Kapitel 5, Seite 220] *Moving Average Filter* genanntes System realisiert werden. Für jedes n -te Sample des Ausgangssignals wird ein Mittelwert aus mehreren Samples in der Umgebung des Samples an Index n des Eingangssignals berechnet. Tiefpassfilter dämpfen die höheren Frequenzen des Spektrums eines Signals. Moving Average Filter erzielen diesen Effekt durch stärkere Glättung schnellerer (höherfrequenter) Veränderungen in den gemittelten Werten im Vergleich zu langsameren (niedrigfrequenter) Veränderungen. In der Signalverarbeitung werden oft Echtzeitberechnungen durchgeführt, daher werden Samples mit kleinerem Index für die Durchschnittswertberechnung genutzt. Bei einer Berechnung des Mittelwerts von 5 Samples gilt

$$y[n] = \frac{1}{5} \sum_{k=0}^4 x[n-k] \iff y[n] = \sum_{k=0}^4 \frac{1}{5} x[n-k]. \quad (3.5)$$

[Lyo10, Kapitel 5, Seite 172].

Die Summenformel zur Berechnung der Ausgangs-Samples dieses Moving Average Filters 3.5 ähnelt der Convolution-Summe 3.4. Auffällige Unterschiede sind endliche Summierung und Verwendung eines festen Koeffizienten pro Summand. Der für jeden Summanden verwendete Koeffizient $\frac{1}{5}$ kann als diskretes Signal, bestehend aus 5 Samples, mit den Werten $\frac{1}{5}$ interpretiert werden:

$$y[n] = \sum_{k=0}^4 h[k]x[n-k] \text{ mit } h = \left(\frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}\right). \quad (3.6)$$

Die Summenformel 3.6 ist eine endliche Convolution-Summe mit vertauschter Indizierung von Eingangs-Signal und Impulsantwort. Nach der kommutativen Eigenschaft der Convolution ist 3.6 äquivalent zu 3.4 [OSB99, Kapitel 2, Seite 29]. So konstruierte Filter werden *Finite Impulse Response Filter*, kurz FIR-Filter, genannt. Die Filterkoeffizienten der Mittelwertberechnung bilden, wie die Notation als h impliziert, die Impulsantwort eines Systems: einem Moving Average Filter. Die Wirkung des Moving Averager Filters wird durch plotten von Ein- und Ausgangssignal anschaulich:

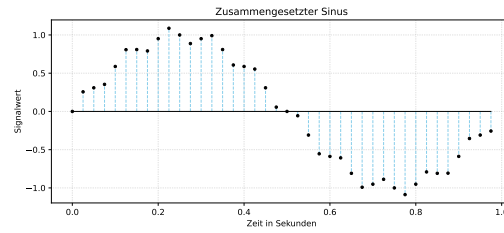


Abbildung 3.1: Ein diskretes Signal, zusammengesetzt aus Sinusschwingungen mit den Frequenzen $f_0 = 1$ Hz, $f_1 = 10$ Hz, und $f_2 = 20$ Hz

Die Anwendung des Moving Averager Filters resultiert in einer deutlich sichtbaren Glättung des Signals, aber auch zu einer Verzögerung des Signals:

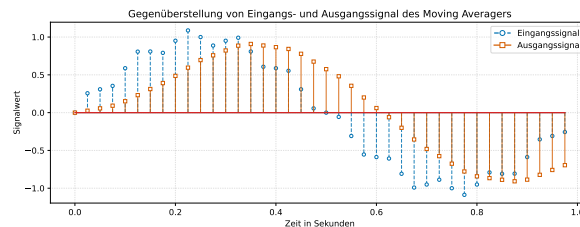


Abbildung 3.2: Das geglättete Signal, über das ungeglättete Signal aus Abbildung 3.1 vorgeblendet

Dieser FIR-Filter ist ein anschauliches Beispiel für den Prozess der Convolution. Die Convolution-Summe selbst ist in allen Anwendungen von Convolution in der Signalverarbeitung gleich. In echten Anwendungsfällen werden oft weit längere Impulsantworten und Signale verwendet. Das resultiert in weit mehr Summanden und Multiplikationen pro berechnetem Ausgangs-Sample. Besonders in Echtzeitanwendungen, in denen eine hinreichend schnelle Berechnung der Ausgangssamples nötig ist, führt die direkte Berechnung von Convolution-Summen zu untragbaren Laufzeiten. Bei der häufigen Audio-Abtastrate von 44,1kHz bleiben für die Berechnung eines Blockes von 256 Ausgangs-Samples weniger als 6 Millisekunden. Für die praktische Anwendung von Convolution zur Abbildung von Systemen wird eine Lösung mit besserem Laufzeitverhalten genutzt. Diese basiert auf dem Convolution-Theorem.

3.1.1 Das Convolution-Theorem

Seien $X(e^{i\omega})$, $H(e^{i\omega})$ und $Y(e^{i\omega})$ die diskreten Fourier-Transformationen des Eingangssignals x , der Impulsantwort h , und des Ausgangssignals y einer Convolution $y[n] = x[n] * h[n]$. Dann gilt:

$$Y(e^{i\omega}) = X(e^{i\omega}) \cdot H(e^{i\omega}). \quad (3.7)$$

Die Convolution zweier Signale in der Zeitdomäne ist äquivalent zu einer Multiplikation der Signale in der Frequenzdomäne. Mit einer inversen diskreten Fourier-Transformation des Ausgangs-Spektrums $Y(e^{i\omega})$ wird das Ausgangssignal in die Zeitdomäne übertragen [OSB99, Kapitel 2, Seite 60]. Bei hinreichend effizienter Berechnung der (inversen) diskreten Fourier-Transformationen kann somit die direkte Berechnung der Convolution-Summe vermieden und durch Multiplikation in der Frequenzdomäne ersetzt werden.

3.2 Laufzeitanalyse

Im Folgenden werden die diskrete Faltung in der Zeitdomäne und die Variante der Faltung, basierend auf der Frequenzdomäne gegenübergestellt.

Dazu betrachten wir zunächst folgenden Pseudocode, der die diskrete Faltung in der Zeitdomäne algorithmisch aufzeigt:

```

1 Algorithm directConvolution(inSignal, N, irSignal, M)
2   outLen <- N + M - 1           // O(1)
3   for i <- 0 to outLen - 1      // O(N + M + 1)
4     for k <- 0 to M - 1        // O(M)
5       if i-k < 0 or n <= i-k  // O(1)
6         continue
7
8       outSignal[i] <- outSignal[i]
9       + irSignal[k] * inSignal[i-k] // Worst-Case: O(1)
10    done
11  done
12  return outSignal

```

Listing 3.1: Grundalgorithmus der Faltung im Zeitbereich

Als Parameter erhält der Algorithmus der direkten Faltung zwei Listen. Die erste Liste enthält die Folge der diskreten Abtastungssignale des Eingangssignals. Dieses Signal hat den Betrag/ die Länge N . Die zweite Liste enthält die diskreten Abtastungssignale der Impulsantwort und hat die Länge M . Insgesamt hat die resultierende Folge eine Länge von $N + M - 1$.

Über diese Länge iteriert dementsprechend die äußere Schleife. Die innere Schleife iteriert über die Länge M der Impulsantwort. Die if-Abfrage der inneren Schleife prüft, ob im Laufe der Iterationen der Index des Eingangssignals 1 unterschreitet. Durch diese Abzweigung an Szenarien ergibt sich eine Worst-Case Laufzeit von

$$\mathcal{O}(1 + (N + M - 1) \cdot (M + 2)) = \mathcal{O}(M^2 + NM - M + 2N - 1) \quad (3.8)$$

Laut Polynomregel fallen Terme niedriger Ordnung weg. Daraus folgt:

$$\mathcal{O}(M^2 + MN) \implies \mathcal{O}(N \cdot M), \text{ denn : } N > M \implies N \cdot M > M^2 \quad (3.9)$$

Die Laufzeitklasse der direkten, diskreten Convolution betragt also laut O-Notation

$$\mathcal{O}(N \cdot M) \quad (3.10)$$

Nun zur Convolution per Multiplikation beider Signale innerhalb der Frequenzdomane. Auch hier der Algorithmus in Pseudocode:

```

1 Algorithm fftConvolution(inSignal, N, irSignal, M)
2   outLen <- N + M - 1 // O(1)
3   fftLen <- outLen    // O(1)
4
5   // zeroPad() fuellt die Listen bis fftLen mit nullen auf
6   zeroPad(inSignal, fftLen) // O(n)
7   zeroPad(irSignal, fftLen) // O(n)
8
9   inSpectrum <- FFT(inSignal, N, fftLen) // O(n log(n))
10  irSpectrum <- FFT(irSignal, M, fftLen) // O(n log(n))
11
12  for f <- 0 to fftLen - 1 // Insg. O(n)
13    outSpectrum[f] <- inSpectrum[f] * irSpectrum[f]
14  done
15
16  outTimeDomain <- IFFT(outSpectrum, fftLen) // O(n log(n))
17
18  // real() isoliert den Realteil des komplexen IFFT-Outputs
19  for i <- 0 to outLen - 1 // Insg. O(n)
20    outSignal[i] <- real(outTime[i])
21  done
22
23  return outSignal

```

Listing 3.2: Grundalgorithmus der Faltung im Zeitbereich

Dieser Algorithmus erhalt als Parameter erneut die beiden Folgen der diskreten Signale des Eingangssignals und der Impulsantwort. Zuerst wird die Lange *fftLen* des Ausgangssignals ermittelt und die beiden Listen mit bis zu dieser Lange mit Nullen aufgefullt. Die Funktion FFT uberfuhrt die beiden Listen jeweils in ihre Frequenzdomane. Anschließend folgt uber die Lange von *fftLen* eine Multiplikation jedes Frequenzwertes.

Die Laufzeitklasse der FFT ist $\mathcal{O}(N \cdot \log(N))$ [Smi97, Kap. 12]. Durch Addition aller anderen Operationen ergibt sich eine Laufzeit von:

$$\mathcal{O}(3(N \cdot \log(N)) + 3N + 2) \in \mathcal{O}(N \cdot \log(N)) \quad (3.11)$$

3.3 LTI-Systeme in der realen Welt

Die Eigenschaften von LTI-Systemen sind in der digitalen Audiotbearbeitung besonders interessant, wo Realweltsysteme als LTI-System interpretiert und durch ihre Impulsantwort approximiert werden können. Ein bekannter Anwendungsfall ist der Convolution Reverb, der das Nachhallverhalten eines Raumes durch Convolution eines Signal mit einer in dem Raum gemessenen Impulsantwort nachstellt. Die Akustik eines Raumes wird durch einen Schallimpuls angeregt, mit einem Mikrofon aufgenommen, und digital gespeichert. Aus dem Messergebnis wird die Impulsantwort des Raums berechnet. Eine Convolution der Impulsantwort mit einem beliebigen Eingangssignal approximiert das Ergebnis die Anwendung der Transferfunktion des Raums auf das Eingangssignal und resultiert in einem Halleffekt, der dem im gemessenen Raum vorhandenen Nachhallverhalten entspricht. Der Raum selbst weist lineare Eigenschaften auf, die eingesetzten Schallwandler und Signalverstärker verursachen aber leichte nonlineare Verzerrungen. Die auf die Schallwandlung folgende Ausbreitung durch die Raumluft ist wiederum nicht vollständig Zeit-Invariant [Far00, Far07a, PSV24]. Durch geeignete Messverfahren in Kombination mit entsprechenden Verfahren zur Berechnung der Impulsantwort (*Deconvolution*) können auch Systeme mit leicht nonlinearen und Zeit-Varianten Eigenschaften vollständig charakterisiert werden [Far00, Far07a]. Neben dem eher spektakulären Anwendungsfall des Convolution Reverbs sind auf Impulsantworten und Convolution basierende Verfahren in der digitalen Signalverarbeitung allgegenwärtig. Sie kommen auch bei der qualitativen Messung von Audio-Equipment, etwa der Frequenzgangbestimmung zum Einsatz. Genauso werden Impulsantworten von Lautsprechersystemen in der Simulation von Gitarrenverstärkern verwendet, um den Einfluss verschiedener an Verstärker angeschlossener Lautsprecherkabinette auf das Signal zu simulieren.

3.4 Messung von Raum-Impulsantworten

Um die akustischen Eigenschaften eines Raumes in Form einer Impulsantwort zu messen, wird der Raum durch einen das gesamte abzubildende Frequenzspektrum umfassenden Messton angeregt [MM01]. Der im Raum resultierende Raumschall wird aufgenommen. Das aufgenommene Signal ist das Ergebnis der Transferfunktion des gemessenen Raumes, angewandt auf den Messton. Da Raumhall-Effekte sich wie in 3.3 dargestellt durch ihre Impulsantwort annähern lassen, kann die IR des Raumes aus diesem Signal berechnet werden [NRH16]. Die Qualität der gemessenen IR ist von der Beschaffenheit des Signals, mit dem der Raum angeregt wird, abhängig. In der Vergangenheit wurden oft pulsive Schallquellen genutzt [Far07b], etwa Schüsse oder platzende Luftballons. Mit der steigenden Verbreitung digitaler Systeme haben sich zunächst die Maximum-Length-Sequence (kurz *MLS*), einem digital erzeugen Rauschen, und die Time Delay Spectrometry (kurz *TDS*), einem linearen Sinus-Sweep, als Messtöne durchgesetzt. Mit diesen Methoden konnten verbesserter Rauschabstand und flachere Frequenzgänge der Impulsantworten erzielt werden [Far07b]. Heutzutage wird vermehrt mit Messverfahren

mit exponentiellen/logarithmischen Sinus-Sweeps (kurz *ESS/LSS*) gearbeitet. Durch die verbesserte Abbildung gemessener Systeme bei Vorhandensein von nichtlinearen oder zeitvarianten Eigenschaften sind ESS/LSS-basierte Verfahren die bevorzugte Messmethode [Far07b].

TDS-basierte Messverfahren nutzen einen Sinus-Sweep, bei dem die Frequenz des Messsignals über die Messdauer linear durch den zu messenden Frequenzbereich verändert wird. TDS-Verfahren sind eher für die Messung kurzer Impulsantworten geeignet, etwa denen von Lautsprecher-Systemen [Far00, HCZ09].

MLS-basierte Messverfahren nutzen periodische Sequenzen von binären Ziffern, die auch als Pseudo-Random Noise bezeichnet werden. Oft werden mehrere Messungen vorgenommen und die Ergebnisse gemittelt. So erzielen MLS-Messungen bessere Rauschabstände als TDS-Messungen [RV89]. MLS-basierte Messverfahren sind jedoch sehr anfällig gegenüber Nonlinearitäten im gemessenen System [HCZ09].

ESS/LSS-basierte Messverfahren nutzen Sinus-Sweeps, bei denen die Frequenz sich in Abhängigkeit der Zeit exponentiell verändert.

Auftretende Artefakte können bei ESS/LSS-Messungen durch ein von [Far00] beschriebenes Deconvolution-Verfahren einfach von der IR getrennt werden. Dabei wird ein zum verwendeten Messton inverses Signal genutzt, welches so berechnet wird, dass eine Convolution des Messtons mit dem inversen Signal einen Einheitsimpuls erzeugt. Convolution dieses inversen Signals mit der Aufnahme des Raumschalls isoliert nicht nur die Impulsantwort, sondern verschiebt auch durch nichtlineare Verzerrungen im Messsystem erzeugte Artefakte in der Zeitdomäne auf einen Zeitpunkt vor der eigentlichen Impulsantwort. Dadurch wird es möglich, diese Artefakte von der eigentlichen Impulsantwort zu trennen, bevor die Impulsantwort zur Nachbildung des Raumhalls verwendet wird [Far00].

Die Anwendung der so gemessenen Impulsantwort auf ein beliebiges Eingangssignal zur Erzeugung eines Convolution Reverbs erfolgt durch Convolution des Eingangssignals mit der Impulsantwort mittels eines der in 3.2 beschriebenen Algorithmen, oder einer für Echtzeitanwendungen angepassten Variante des FFT-basierten Convolution-Algorithmus.

4 Digitale Audiotbearbeitung:

Implementierung der Convolution

4.1 Einleitung

In der folgenden Sektion wir nun das Wissen aus den vorherigen Kapiteln genutzt und eine Convolution zweier Audiosignale durchgeföhrt. Daför wird die die zuvor vorgestellte Gleichung der diskreten Convolution angewendet [3.1](#):

$$y[n] = \sum_{k=0}^{M-1} h[k] \cdot x[n - k] \quad (4.1)$$

Wobei:

n : Folgenindex (Maximum ist Länge des Ausgangssignals)

M : Länge der Impulsantwort

För die konkrete Umsetzung als Programm wird hier die Sprache *C++* genutzt. Es sollen zuerst die diskreten Abtastwerte der jeweiligen Audiodateien in Arrays geladen werden. Danach soll die Convolution durchgeföhrt und anschließend das Ergebnis als neue Audiodatei im Hintergrundspeicher abgelegt werden.

För das Laden und speichern von Audiodateien im .wav-Format wird hier das bekannte Framework *JUCE* genutzt [?].

4.2 Implementierung in C++

Der folgende Quellcodeausschnitt zeigt eine Implementierung der direkten, diskreten Convolution nach Summenformel: [4.1](#)

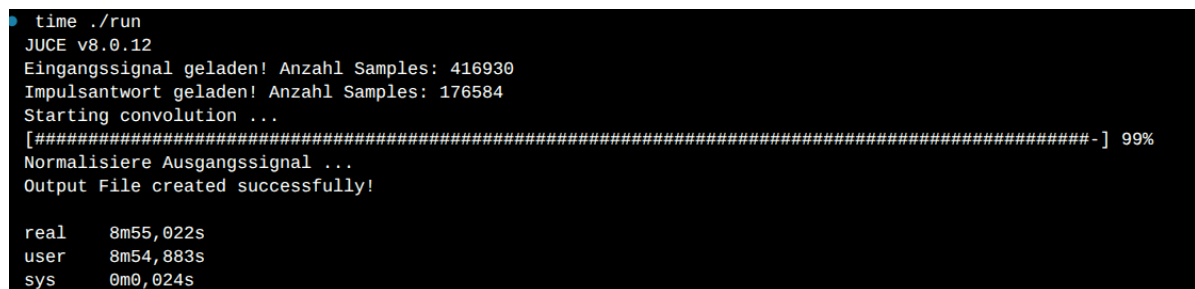
```
1 std::vector<float> convolution(std::vector<float>& input, std::  
    vector<float>& ir) {  
2  
3     unsigned long inputLen {input.size()};  
4     unsigned long irLen {ir.size()};  
5     unsigned long outputLen {inputLen + irLen - 1};
```

```
6
7  std::vector<float> output (outputLen, 0.0f); //<--
   Initialisiere vector mit der Pufferlaenge = outputLen und 0.0
   f als Initialwerte
8
9  for (int i = 0; i < outputLen; i++) {
10     for (int k = 0; k < irLen; k++) {
11         if (i - k < 0 || inputLen <= i - k) {
12             continue;
13         }
14         output[i] += ir[k] * input[i - k];
15     }
16 }
17
18 return output;
19 }
```

Listing 4.1: Grundalgorithmus der Convolution im Zeitbereich

Als Parameter werden in Listing 4.1 zwei Listen, nämlich jeweils die Folgen an normalisierten Abtastwerten des Eingangssignals und der Impulsantwort übergeben. Dadurch, dass die Beträge/Längen der Folgen bekannt sind, lässt sich in Zeilen 5 bis einschl. 8 im Voraus die Länge der Ausgangssignalfolge nach $|y[n]| = |h[n]| + |x[n]| - 1$ berechnen und die Werte mit 0 initialisieren. Schließlich wird die Formel 4.1 in Form der verschachtelten Schleife in Zeilen 9 bis 16 umgesetzt. In manchen Schleifeniterationen kommt es dazu, dass der Index von $x[n - k]$ kleiner als null wird oder die Länge des Eingangssignals übersteigt. Diese Fälle werden in Zeile 11 geprüft und ggf. wird die jeweilige Iteration der Schleife übersprungen.

Diese Vorgehensweise ist unter Kenntnis der diskreten Gleichung der Convolution 4.1 leicht nachzuvollziehen, weil sie hier direkt in Code umgesetzt wird. Es gibt aber hinsichtlich der Nutzung in der Praxis ein großes Problem, das durch das folgende Bildschirmfoto unterstrichen wird:



```
time ./run
JUICE v8.0.12
Eingangssignal geladen! Anzahl Samples: 416930
Impulsantwort geladen! Anzahl Samples: 176584
Starting convolution ...
[#####-] 99%
Normalisiere Ausgangssignal ...
Output File created successfully!

real    8m55,022s
user    8m54,883s
sys      0m0,024s
```

Abbildung 4.1: Laufzeitmessung des Algorithmus 4.1 unter Verwendung des Linux-Kommandos *time*

Das Ergebnis der Messung, die in Abbildung 4.1 gezeigt wird, bezieht sich auf die Convolution eines 10-sekündigen Gitarrenssignals (416.960 Abtastungen) mit einer Impulsantwort von 5 Sekunden (176.584 Abtastungen). Ergebnis ist ein Halleffekt auf der Gitarre. Wie an der Messung zu sehen, dauerte die Berechnung des 15-sekündigen Resultats fast ganze 9 Minuten an. Diese Dauer ergibt in Anbetracht der zweifach verschachtelten Schleife und der Menge an Abtastwerten durchaus Sinn. Schaut man sich im Quellcode 4.1 die Anzahl der Schleifendurchläufe, jew. von innerer und äußerer Schleife an, dann ergibt sich folgende Laufzeitklasse:

$$\mathcal{O}(N \cdot M) \approx \mathcal{O}(n^2) \quad (4.2)$$

N sei hierbei die Länge des Ausgangssignals und M die Länge der Impulsantwort

Das entspricht speziell in diesem Experiment ausmultipliziert 104.805.076.176 Berechnungen, die der Prozessor also für ein 15-sekündiges Audiosignal ausführen muss. Eine solche Laufzeit ist für den Gebrauch in der Praxis nicht hinnehmbar und der Grund, weshalb Berechnungen der diskreten Convolution lange Zeit ausschließlich in der Forschung ihren Platz fanden [Lyo10, Kap. 4].

4.3 Fazit und Ausblick auf FFT

Wie am Experiment in der vorherigen Sektion 4.2 gesehen, ist eine direkte Implementierung der diskreten Convolution 4.1 in Audiosoftware und erst recht in Echtzeitanwendungen nicht sinnvoll.

Als Ausblick sei jedoch die Implementierung der Convolution mithilfe der *Fast Fourier Transformation* (kurz: *FFT*) erwähnt. Diese hat, im Gegensatz zur diskreten Fourier Transformation eine schnelle Laufzeit von $\mathcal{O}(N \cdot \log N)$ [Smi97, Kap. 12]. Mithilfe dieses Verfahrens lassen sich beide Signale in ihre Frequenzdomänen umwandeln. Weil in der Frequenzdomäne die Convolution lediglich einer einfachen Multiplikation entspricht, lässt sie sich dort extrem schnell durchführen. Anschließend folgt durch die Inverse der FFT die Umwandlung des Ergebnisses zurück in die Zeitdomäne. Erst dieses Vorgehen macht eine Nutzung der Convolution in der Praxis effizient möglich.

Literaturverzeichnis

- [Far00] Angelo Farina. Simultaneous measurement of impulse response and distortion with a swept-sine technique. February 2000.
- [Far07a] Angelo Farina. Advancements in impulse response measurements by sine sweeps. 2007.
- [Far07b] Angelo Farina. Impulse Response Measurements. 2007. URL: <https://www.angelfarina.it/Public/papers/238-NordicSound2007.pdf>.
- [HCZ09] Martin Holters, Tobias Corbach, and Udo Zölzer. Impulse Response Measurement Techniques and their Applicability in the Real World. In *Proc. of the 12th Int. Conference on Digital Audio Effects*, Como, Italy, September 2009.
- [Lyo10] Richard G. Lyons. *Understanding Digital Signal Processing*. Pearson Education, third edition, 2010.
- [MM01] Swen Müller and Paulo Massarani. Transfer Function Measurement with Sweeps. *Journal of the Audio Engineering Society*, 49(6):443–471, June 2001. URL: <https://www.aes.org/e-lib/browse.cfm?elib=10189>.
- [NRH16] Antonin Novak, Frantisek Rund, and Petr Honzik. Impulse Response Measurements using MLS Technique on Nonsynchronous Devices. *Journal of the Audio Engineering Society*, 64(12):978–987, December 2016. URL: <http://www.aes.org/e-lib/browse.cfm?elib=18532>, doi:10.17743/jaes.2016.0050.
- [OSB99] Alan V. Oppenheim, Ronald W. Schaffer, and John R. Buck. *Discrete-time signal processing*. Prentice Hall, Upper Saddle River, N.J, 2nd ed edition, 1999.
- [PSV24] K. Prawda, S. Schlecht, and V. Välimäki. Time Variance in Measured Room Impulse Responses. In *Proceedings of the 10th Convention of the European Acoustics Association Forum Acusticum 2023*, pages 1639–1646, Turin, Italy, January 2024. European Acoustics Association. doi:10.61782/fa.2023.0398.
- [RV89] Douglas D Rife and John Vanderkooy. Transfer-Function Measurement with Maximum-Length Sequences. *Journal of the Audio Engineering Society*, 37(6):419–444, 1989.
- [Smi97] Steven W. Smith. *The Scientist & Engineer’s Guide to Digital Signal Processing*. California Technical Pub, first edition, 1997.